

Practical No. 2

Aim: Install a C compiler in the virtual machine created using virtual box and execute simple programs.

Software Required: Oracle VirtualBox, Ubuntu Linux virtual machine.

Theory:

Cloud computing is based on virtualization technology, which enables multiple operating systems to run on a single physical machine. VirtualBox is a virtualization tool that allows the creation of virtual machines, providing a cloud-like environment for software execution.

In Linux systems, the **build-essential** package provides the **GCC (GNU Compiler Collection)**, which includes the **C compiler (gcc)**. The gcc compiler translates C source code into machine-executable format. Installing and executing C programs inside a virtual machine demonstrates program execution in a virtualized cloud environment.

Procedure:

Step 1: Open the Terminal using `ctrl + alt + T` shortcut.

Step 2: Update the package repository using the command: `sudo apt update`

```
vboxuser@ubuntu:~$ sudo apt update
[sudo] password for vboxuser:
Hit:1 http://in.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:3 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease
Get:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [208 B]
Get:7 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [71.5 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [208 B]
Fetched 220 kB in 5s (46.0 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
151 packages can be upgraded. Run 'apt list --upgradable' to see them.
vboxuser@ubuntu:~$
```

Step 3: Install the C Compiler, using the command: `sudo apt install build-essential`

```
vboxuser@ubuntu:~$ sudo apt install build-essential
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu bzip2 dpkg-dev fakeroot g++ g++-13 g++-13-x86-64-linux-gnu
  g++-x86-64-linux-gnu gcc gcc-13 gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu libalgorithm-diff-perl
  libalgorithm-diff-xs-perl libalgorithm-merge-perl libasan8 libbinutils libbcc1-0 libctf-nobfd0 libctf0 libdpkg-perl
  libfakeroot libfile-fcntllock-perl libgcc-13-dev libgprofng0 libhwasan0 libitm1 liblsan0 libquadmath0 libsframe1
  libstdc++-13-dev libtsan2 libubsan1 lto-disabled-list make
Suggested packages:
  binutils-doc gprofng-gui bzip2-doc debian-keyring g++-multilib g++-13-multilib gcc-13-doc gcc-multilib autoconf
  automake libtool flex bison gcc-doc gcc-13-multilib gcc-13-locales gdb-x86-64-linux-gnu git bzr libstdc++-13-doc
  make-doc
The following NEW packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu build-essential bzip2 dpkg-dev fakeroot g++ g++-13
  g++-13-x86-64-linux-gnu gcc gcc-13 gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu libalgorithm-diff-perl
  libalgorithm-diff-xs-perl libalgorithm-merge-perl libasan8 libbinutils libbcc1-0 libctf-nobfd0 libctf0
  libdpkg-perl libfakeroot libfile-fcntllock-perl libgcc-13-dev libgprofng0 libhwasan0 libitm1 liblsan0
  libquadmath0 libsframe1 libstdc++-13-dev libtsan2 libubsan1 lto-disabled-list make
```

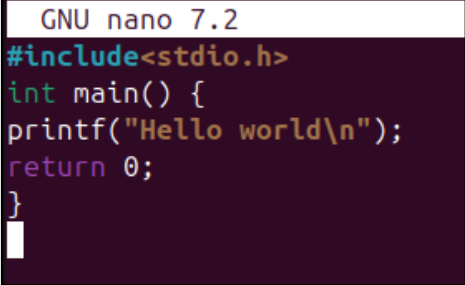
Step 4: Verify C Compiler installation, using the command: `gcc -version`

```
vboxuser@ubuntu:~$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Step 5: Create a C program file, using: `nano hello.c`

Program Code:

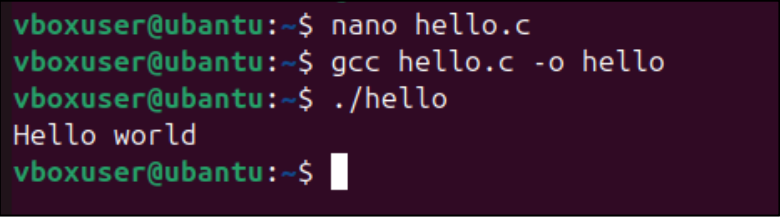
```
#include <stdio.h>
int main() {
    printf("Hello world!\n");
    return 0;
}
```

A screenshot of the GNU nano 7.2 text editor. The title bar at the top says "GNU nano 7.2". The editor content shows the C program code: `#include<stdio.h>`, `int main() {`, `printf("Hello world\n");`, `return 0;`, and `}`. A white cursor is visible at the end of the closing brace on the last line.

Step 6: Compile the C program, by typing: `gcc hello.c -o hello`

Step 7: Execute the program, by using the command: `./hello`

Output: Hello world!

A screenshot of a terminal window showing the steps to compile and run the program. The commands and their outputs are: `nano hello.c` (no output), `gcc hello.c -o hello` (no output), `./hello` (output: `Hello world`), and a final prompt `vboxuser@ubuntu:~$` with a cursor.

Conclusion: Hence, we have successfully installed a C compiler in the virtual machine created using virtual box and executed a simple program.